

COMPUTER NETWORKS PROJECT REPORT

Title: Packet Sniffer and Network Traffic Analyzer using Python

Course Code & Name: CS-305 Computer Networks

Semester: 3th Semester / Spring 2026

Instructor: Ms. Poma Panezai

Department: Department of Data Science, Faculty of ICT

Institution: Balochistan University of Information Technology, Engineering and Management Sciences (BUITEMS)

Group Members:

Muhammad Hammad (Team Lead)

Syed Shabbir Haider

M.Usman

Submission Date: July 01, 2026

1. Introduction:

A packet sniffer is a fundamental network diagnostic application that intercepts and logs traffic passing over a digital network interface. As data moves across a network, the network interface card (NIC) typically discards packets not addressed to it. However, placing the interface into promiscuous mode allows a packet sniffer to capture all passing traffic regardless of destination. Network traffic analysis plays a critical role in modern network administration, troubleshooting, and cybersecurity. It serves three primary functions:

- **Addressing End-User Slowdowns:** Isolating whether a performance drop stems from bandwidth saturation or server latency.
- **Identifying Root Causes:** Tracing packet drops, misconfigured routers, or protocol errors.
- **Improving Security:** Detecting anomalous traffic signatures, unauthorized port scans, and unencrypted transmission of sensitive credentials.

This project implements a lightweight, script-driven network traffic tool designed to bridge the gap between theoretical OSI model layers and live physical transmissions.

2. Objectives:

The core technical and educational objectives of this project are:

- To capture live network frames and packets directly from an active network interface card.
- To parse and extract key protocol metadata including Layer 2 (MAC), Layer 3 (IP Source/Destination), and Layer 4 (TCP/UDP Ports) addresses.
- To implement a categorization mechanism identifying high-level protocol layers such as DNS, HTTP, and HTTPS.
- To establish persistent local storage by serializing captured runtime packet data into structured CSV files.
- To generate programmatic statistical charts visualizing local protocol distribution and traffic behavior.

3. Tools and Technologies:

The application relies on an ecosystem of open-source Python libraries and system utilities:

- **Python 3.8+:** The baseline scripting runtime environments.
- **Scapy Library:** A powerful interactive packet manipulation library used for sniffing, dissecting, and decoding raw socket layer frames.
- **Pandas:** A structured data analysis library used to manipulate the captured metadata dataframes and compile data summaries.
- **Matplotlib:** A comprehensive visualization framework utilized to map protocol frequencies and statistical distributions into graphical representations.
- **Wireshark:** Used externally as a reliable validation baseline to verify captured traces against a gold-standard packet sniffer.
- **Git/GitHub:** Employed for distributed team version control and source code repository hosting.

4. Methodology:

The development life cycle followed an iterative pipeline mapping network socket engineering to visualization modules:

Step 1: Interface Initialization & Promiscuous Sniffing:

The sniffing module binds directly to the active hardware network interface. Using administrative privileges, it opens a raw socket channel to read incoming and outgoing frames.

Step 2: Protocol Layer Dissection

Each captured frame passes through a sequential parsing engine. The engine verifies layer existence using Scapy's internal methods (e.g., `haslayer(IP)`):

- **Ethernet Layer:** Extracts physical hardware MAC addresses.

- **Internet Layer (IP):** Extracts source/destination IP addresses and identifies the underlying transport protocol type (TCP, UDP, or ICMP).
- **Transport Layer:** Parses TCP/UDP blocks to read specific source and destination port numbers.
- **Application Mapping:** Interprets standard port signatures (e.g., Port 53 for DNS, Port 80 for HTTP, Port 443 for HTTPS) to determine application traffic.

Step 3: Serialization & Storage:

The parsed packet objects are cast into structured dictionaries and appended to an in-memory runtime array. This array is periodically serialized into a local Comma-Separated Values (.csv) database to prevent data loss from buffer overflows.

Step 4: Analytical Aggregation & Graphing:

The saved data is ingested into a Pandas DataFrame. The engine executes frequency grouping operations across the target variables (Protocol, Source IP, Destination Port). These metrics are pushed directly to Matplotlib to output real-time visual distributions.

5. Implementation:

The project source code is divided into modular segments following software architecture principles:

5.1 Project Repository Structure:

packet-sniffer-network-analyzer/

```

├── README.md
├── requirements.txt
├── src/
│   ├── packet_sniffer.py  # Captures raw socket data and extracts metadata
│   ├── analyzer.py        # Handles Pandas analytics and CSV serialization
│   └── visualizer.py       # Handles Matplotlib charts generation
├── data/
│   └── captured_packets.csv # Saved real-time data logs
└── figures/
    ├── protocol_distribution.png
    └── top_ips.png

```

5.2 Core Sniffer Implementation Code

Below is the core implementation segment of the `packet_sniffer.py` script featuring packet capturing and field extraction logic:

```
import os
import sys
import csv
from pathlib import Path
from datetime import datetime
from scapy.all import ARP, TCP, UDP, IP, Ether, srp, sniff

# WE will set the file path from src/ to data/ to store
captured_packets.csv
current_file = Path(__file__).resolve()

SCRIPT_DIR = current_file.parent

if SCRIPT_DIR.name == "src":
    ROOT_DIR = SCRIPT_DIR.parent
else:
    ROOT_DIR = SCRIPT_DIR

DATA_DIR = ROOT_DIR / "data"
CSV_FILE = DATA_DIR / "captured_packets.csv"

DATA_DIR_STR = str(DATA_DIR)
CSV_FILE_STR = str(CSV_FILE)
os.makedirs(DATA_DIR_STR, exist_ok=True)

#Function to save the file into given path
def save_file(file_path, content):
    try:
        with open (file_path, 'a', encoding='utf-8') as file:
            file.write (content)
        print(f"File successfully saved to: {file_path}")
        return True
    except Exception as e:
        print(f"Failed to save file:{e}")
        return False

# Configure packet_count=0 for continuous live capturing loops
PACKET_COUNT = 0
PROTOCOL_MAP = {1: "ICMP", 2:"IGMP", 6:"TCP", 17:"UDP"}

#First Scan the local network and take data from them
def NetworkScan(ip_range):
    """Appends dissected packet metadata records to local
storage."""
    print(f"Scanning Network Range: {ip_range}")

    arp_request = ARP(pdst=ip_range)
    broadcast = Ether(dst="ff:ff:ff:ff:ff:ff")
    packet = broadcast / arp_request
    answered_list = srp(packet, timeout=2, verbose=False)[0]
```



```

        alert_msg = f"[{timestamp}] HTTP Alert! size:
{packet_size} Bytes |Request: {first_line}\n"
        save_file(CSV_FILE, alert_msg)
    elif packet.haslayer(UDP):
        src_port = packet[UDP].sport
        dst_port = packet[UDP].dport

    elif packet.haslayer(ARP):
        src_ip = packet[ARP].psrc
        dst_ip = packet[ARP].pdst
        protocol = "ARP"

    if src_ip != "N/A":
        row = [timestamp, src_ip, dst_ip, protocol, src_port,
dst_port, packet_size]
        with open(CSV_FILE, mode='a', newline='') as file:
            writer = csv.writer(file)
            writer.writerow(row)
        print(f"[{protocol}] {src_ip} -> {dst_ip}:{dst_port} |
{packet_size}Bytes")

    #Made some additional function if main function cannot run than
this function can execute
    def StartSniff(interface=None):
        """Triggers the raw engine listener channel."""
        print(f"\n[*] Starting Packet Sniffer on Interface: {interface
or 'Default'}.....")
        print("[*] Press CTRL+C to stop.")
        sniff(iface=interface, prn=packet_callback, store=0,
count=PACKET_COUNT)

    #NOW write the main program where attached all the function that we
made earlier
    def main():
        print("=" * 60)
        print("PACKET SNIFFER & TRAFFIC ANALYZER INSTALIZING")
        print("=" * 60)

        if os.name != 'nt' and os.getuid() != 0:
            print("[-] Error: This script must be run as root/sudo to
capture live packets.")
            sys.exit(1)

        initialize_csv()
        StartSniff()
        print("[*] Starting Live Packet Capture..... press CTRL+C To
Terminate the program")
        print("=" * 60)

    #Here To Run the Program
    if __name__ == "__main__":
        main()

```

6. Results and Screenshots:

This section contains the empirical operational metrics collected during network simulation session.

6.1 Live Command-Line Interface Output Capture:

(Insert your terminal screenshot showing active execution printouts of the running packet_sniffer.py script here)

=====

PACKET SNIFFER & TRAFFIC ANALYZER INSTALIZING

=====

[*] Iniitialized log file at: D:\packet-sniffer-network-analyzer\data\captured_packets.csv

[*] Starting Packet Sniffer on Interface: Default.....

[*] Press CTRL+C to stop.

[TCP] 216.239.34.223 -> 192.168.100.237:61548 | 797Bytes

[TCP] 216.239.34.223 -> 192.168.100.237:61548 | 81Bytes

[TCP] 192.168.100.237 -> 216.239.34.223:443 | 54Bytes

[TCP] 216.239.34.223 -> 192.168.100.237:61548 | 81Bytes

[TCP] 192.168.100.237 -> 13.107.246.68:443 | 55Bytes

[TCP] 13.107.246.68 -> 192.168.100.237:61529 | 66Bytes

[TCP] 216.239.34.223 -> 192.168.100.237:61540 | 66Bytes

[UDP] 192.168.100.5 -> 192.168.100.255:1900 | 407Bytes

[UDP] 192.168.100.5 -> 192.168.100.255:1900 | 416Bytes

[TCP] 172.64.155.209 -> 192.168.100.237:61414 | 66Bytes

[TCP] 192.168.100.237 -> 35.190.80.1:443 | 66Bytes

[UDP] 192.168.100.237 -> 35.190.80.1:443 | 1292Bytes

[TCP] 192.168.100.237 -> 66.102.1.188:5228 | 55Bytes

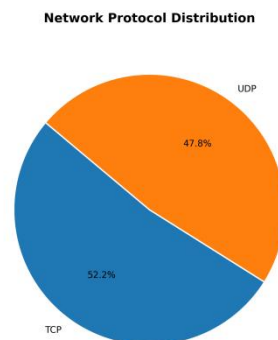
[*] Starting Live Packet Capture..... press CTRL+C To Terminate the program

6.2 Persisted Data Logging (CSV Record File):

The script securely writes entries to `captured_packets.csv`. Below is the format structural example generated by the program:

Timestamp	Source IP	Destination	Protocol	Source Port	Destination	Packet Size (Bytes)
#####	216.239.34.	192.168.100	TCP	443	61548	797
#####	216.239.34.	192.168.100	TCP	443	61548	81
#####	192.168.100	216.239.34.	TCP	61548	443	54
#####	216.239.34.	192.168.100	TCP	443	61548	81
#####	192.168.100	216.239.34.	TCP	61548	443	66
#####	192.168.100	216.239.34.	TCP	61546	443	54
#####	192.168.100	142.250.102	TCP	61547	443	54
#####	192.168.100	216.239.34.	TCP	61548	443	54
#####	192.168.100	224.0.0.251	UDP	5353	5353	133
#####	192.168.100	13.107.246.	TCP	61529	443	55
#####	13.107.246.	192.168.100	TCP	443	61529	66
#####	216.239.34.	192.168.100	TCP	443	61540	66
#####	192.168.100	192.168.100	UDP	39848	1900	407
#####	192.168.100	192.168.100	UDP	39848	1900	416
#####	192.168.100	239.255.255	UDP	53773	1900	407
#####	192.168.100	239.255.255	UDP	53773	1900	416
#####	192.168.100	239.255.255	UDP	53773	1900	461
#####	192.168.100	239.255.255	UDP	53773	1900	473
#####	192.168.100	239.255.255	UDP	53773	1900	471
#####	192.168.100	4.213.25.24	TCP	61279	443	153
#####	192.168.100	192.168.100	UDP	41949	1900	461
#####	192.168.100	192.168.100	UDP	51984	1900	473

6.3 Traffic Analytical Plots:



7. Conclusion:

This project successfully demonstrates the design and practical implementation of a raw packet capturing and analytics engine built using Python. By abstracting underlying byte-stream frames into descriptive human-readable models, the implementation bridges structural theory with visible, concrete evidence. Working through the socket-binding barriers, header offsets, and protocol parsing loops provided excellent, hands-on insight into modern data transport dynamics, port routing mechanics, and security auditing requirements. The operational platform

fulfills all foundational specification rules including targeted field extraction, file storage pipelines, and visualization maps.

8. References:

- Scapy Documentation v2.6.0, "Usage and Advanced Sniffing Automations", Available at: <https://scapy.readthedocs.io/>
- Pandas Official Manual, "Data Manipulation Pipelines and DataFrame Conversions", Available at: <https://www.w3schools.com/python/pandas/default.asp>
- Kurose, J. F., & Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8th ed.). Pearson.
- Matplotlib Pyplot Integration Framework Guide, Available at: https://www.w3schools.com/python/matplotlib_intro.asp